
SPACE FACULTY
Let's learn about satellites!
Space mission and satellite design course
Satellite Command & Data Handling
2NDSpace

Managing Data and Tasks Beyond Earth

"Satellites are just like your smartphone, without the touchscreen and someone telling them what to do"

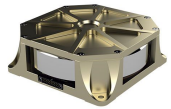
Why is data management crucial in space?

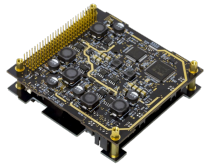
Mission failures due to data systems problems:

- Mars Climate Orbiter (1999)
- Mars Pathfinder (1997)
- Galileo (1995)
- Chandrayaan-2 (2019)
- Galileo (2003)

Satellite Data System Overview

- Communication Systems (Transceivers, Antennas)
- Power Management and Distribution (PMAD)
- Solar Panels Electronics (EPS)
- Sensors (Temperature, Accelerometers, etc.)
- Reaction Wheels and Magnetorquers
- Thermal Control Systems
- Navigation Systems (e.g., GPS)
- Payload Instruments (Mission-specific)





OBDH & OBC

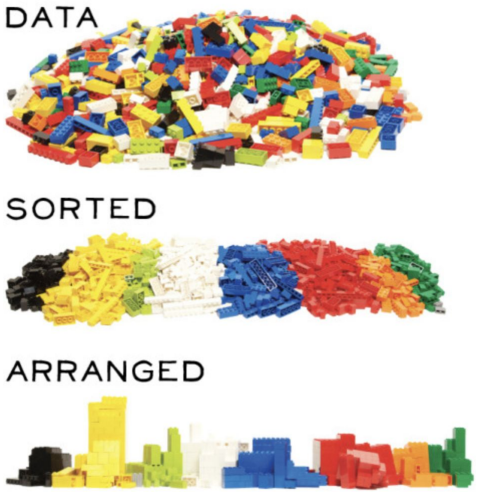
On Board Data Handling (OBDH)

- Manages data: Receives, stores, and transfers data efficiently.
- Focuses on data management and telemetry.
- Less autonomy, follows predefined protocols.

On Board Computer (OBC)

- Handles computation: Executes software, controls spacecraft, and performs calculations.
- Focuses on spacecraft control, navigation, and mission-specific tasks.
- Higher autonomy, involves decision-making in dynamic conditions.

OBDH: The Librarian



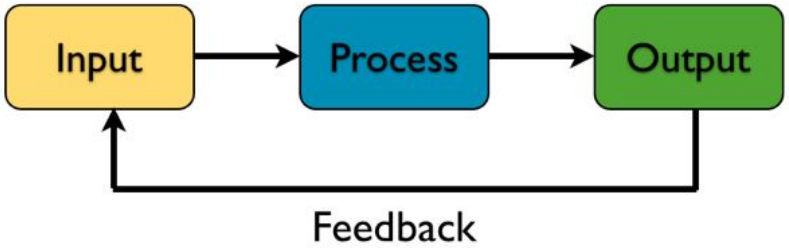
The OBDH can be compared to the librarian in a library. The librarian is responsible for organizing, cataloging, and managing books and information in the library.

Role in Spacecraft: OBDH focuses on the reception, storage, and management of data generated by various spacecraft systems and sensors. It ensures that data is organized, stored efficiently, and prepared for transmission to ground control.

OBC: The Researchers

The OBC is like a team of researchers in the library. Researchers use the library's resources to conduct experiments, analyze data, and make decisions based on their findings.

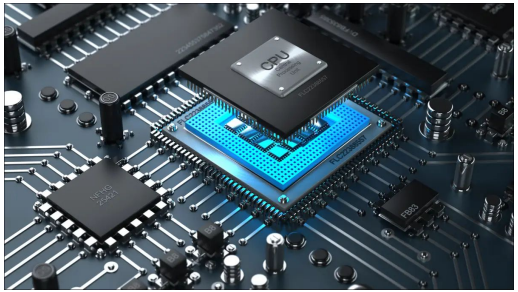
Role in Spacecraft: The OBC Utilizing the data organized by OBDH. It processes data, executes software, and makes decisions to control the spacecraft's behavior, navigate, and accomplish mission tasks.



OBDH and OBC Interplay

The collaboration between the librarian and the researchers represents the interplay between OBDH and OBC. The librarian ensures that information is readily available and well-organized, while the researchers use that information to achieve specific goals.

Role in Spacecraft: In a similar way, OBDH and OBC work together to ensure the spacecraft's success. OBDH manages data so that OBC can use it effectively for spacecraft control, navigation, and mission execution.



Dimensional Aspects

- CubeSat Form Factor: Components must fit within standard CubeSat sizes (e.g., 1U, 2U, 3U, 6U, 12U).
- CubeSat Frame and Structure: Design should fit standard mounting rails and attachment points.
- Size of Processing Units: Compact, low-power processors are preferred.
- Data Storage and Memory: Choose memory and storage that fit the form factor.
- Payload and Sensors: Account for size and configuration of mission-specific sensors.
- Wiring and Cabling: Plan for efficient wiring and connectivity.
- Thermal Considerations: Allocate space for thermal management components.

Main Solutions for OBDHs

- Flash Memory: Flash memory chips are used for non-volatile storage of mission-critical data and software. They offer reliability and low power consumption.
- Solid-State Drives (SSDs): SSDs provide high-speed, non-volatile data storage for OBDH systems. They are used in missions with large data storage requirements.
- Data Compression Algorithms: Data compression techniques reduce the size of stored or transmitted data, optimizing limited storage and bandwidth resources.
- Telemetry Systems: Telemetry systems collect and transmit spacecraft health and status data to ground control. They include sensors, transmitters, and antennas.
- Redundancy and Error Detection: Redundancy techniques ensure mission reliability by incorporating backup components and error-detection mechanisms.
- Data Routing and Switching: OBDH systems include components for routing and switching data between various subsystems and instruments on the spacecraft.
- Thermal Management: Thermal solutions are essential to prevent overheating and ensure OBDH components operate within their specified temperature ranges.

Main Solutions for OBDHs

When OBDH is Necessary:

- When the spacecraft generates, receives, and processes a significant amount of data from various sensors, instruments, or payloads.
- For missions that require a high degree of autonomy, such as Earth observation, remote sensing, or interplanetary exploration.
- In missions with limited bandwidth for data transmission, OBDH can include data compression algorithms that reduce the amount of data sent to Earth, optimizing data transfer.
- When the mission has a low fault tolerance, OBDH can implement error detection and correction mechanisms, enhancing spacecraft reliability.
- OBDH is crucial for collecting and managing telemetry data, which provides information about the spacecraft's health and status.

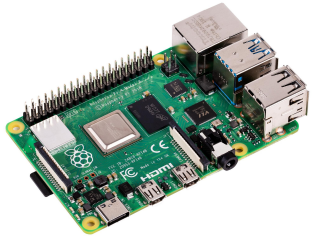
Main Solutions for OBDHs

When OBDH May Not Be Necessary:

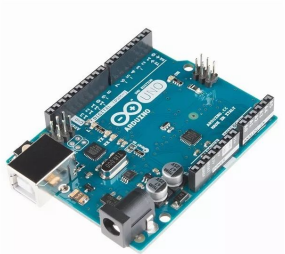
- For very simple missions with minimal data requirements, such as CubeSats with basic functions like transmitting simple messages or collecting a small amount of data, an OBDH subsystem may be minimal or unnecessary.
- In missions where the spacecraft primarily serves as a communication relay (e.g., a satellite in geostationary orbit), data handling requirements may be less complex, and OBDH can be simplified.
- Missions with minimal data generation and transmission needs may rely on simpler data handling methods without a dedicated OBDH subsystem.

Main Solutions for OBCs

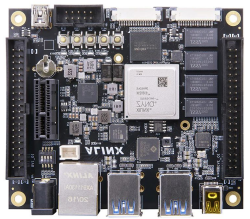
Single-Board Computers (SBCs): These are compact, integrated computers with processors, memory, and I/O interfaces on a single board. SBCs are commonly used in spacecraft for their simplicity and reliability.



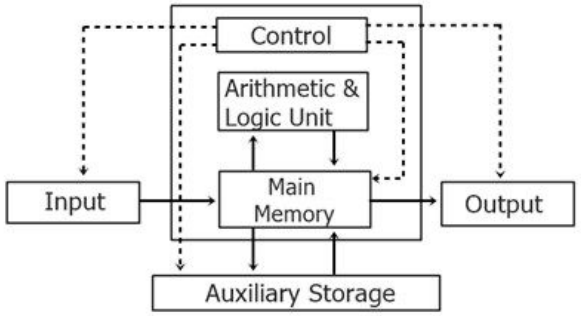
Microcontrollers: Microcontrollers are low-power, embedded systems used for specific tasks like sensor interfacing and basic control functions. They are often employed in CubeSats.



Field-Programmable Gate Arrays (FPGAs): FPGAs offer flexibility in hardware acceleration and real-time processing. They are used for tasks requiring high-speed data processing and custom logic.



Single-Board Computers



Block Diagram of Computer

Typical Architecture

Central Processing Unit (CPU):
- Integrated CPU with an Arithmetic Logic Unit (ALU) for data processing.
- Executes software instructions and algorithms for mission-critical tasks.

Memory:
- Includes Random Access Memory (RAM) for temporary data storage.
- Utilizes Flash memory or Solid-State Drives (SSDs) for data and program storage.

Input/Output (I/O) Interfaces:
- Interfaces for communication with sensors, instruments, and subsystems.
- Facilitates data exchange and command execution.

Operating System (OS):
- Runs specialized embedded operating systems for software execution.
- Provides a platform for mission-specific software applications.

Software and Algorithms:
- Hosts software applications and algorithms tailored to the mission's needs.
- Manages data processing, control, and communication tasks.

Microcontrollers

Typical Architecture

Microcontroller Unit (MCU):

- Contains an integrated CPU core with a simplified Arithmetic Logic Unit (ALU).
- Designed for low-power and specific embedded tasks.

Memory:

- Embedded Flash memory for program storage.
- RAM for temporary data storage during operations.

Input/Output (I/O) Peripherals:

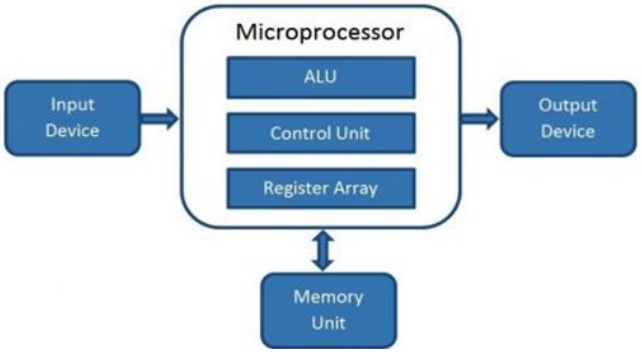
- Interfaces with sensors, actuators, and external components.
- Collects sensor data and manages control functions.

Analog-to-Digital Converter (ADC):

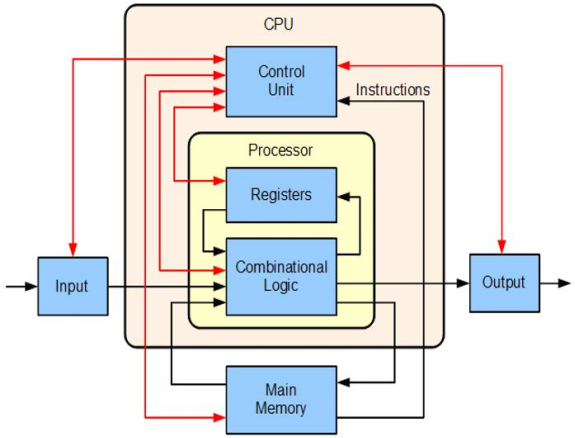
- Converts analog sensor data into digital form for processing.
- Essential for interfacing with analog sensors.

Timers and Counters:

- Used for timing and scheduling tasks, making microcontrollers suitable for real-time applications.



FPGAs



Typical Architecture

Configurable Logic Blocks (CLBs):

- An array of reconfigurable logic blocks that can be programmed to implement custom digital logic circuits.
- Facilitates the creation of specialized hardware.

Input/Output (I/O) Blocks:

- Provides interfaces for external connections, enabling data flow in and out of the FPGA.
- Supports communication with sensors and subsystems.

Interconnect Resources:

- Programmable interconnects that route signals between CLBs and I/O blocks.
- Enables flexible connectivity between logic elements.

Memory:

- Stores the bitstream configuration file, defining the custom logic implemented in the FPGA.
- Allows for reprogramming as needed.

Comparing SBCs, Microcontrollers and FPGAs

Similarities:

- Processing Capabilities: All three execute instructions, run software, and process data.
- Memory: Each has storage components for data and programs.
- Input/Output (I/O): Equipped with interfaces for external communication.
- Customization: Configurable for mission-specific tasks.

Differences:

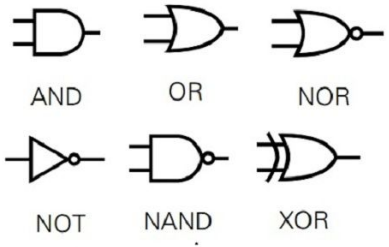
- Complexity: SBCs: Versatile, run diverse software. Microcontrollers: Simpler, for specific tasks. FPGAs: Customizable hardware logic.
- Processing Speed: SBCs: High processing speeds. Microcontrollers: Lower processing speeds, real-time. FPGAs: Real-time, logic-dependent.
- Power Consumption: SBCs: Higher power consumption. Microcontrollers: Low-power for extended missions. FPGAs: Variable based on configuration.
- Reconfigurability: SBCs: Software updates. Microcontrollers and FPGAs: Reprogrammable.
- Use Cases: SBCs: General computing, complex tasks. Microcontrollers: Low-power, sensor interfacing. FPGAs: Custom hardware, signal processing.

Comparing SBCs, Microcontrollers and FPGAs

Another fundamental difference is their **computational approach**. FPGAs rely on Boolean algebra to create custom logic circuits, whereas SBCs and microcontrollers primarily utilize binary arithmetic within software applications.

BOOLEAN

Boolean algebra is a mathematical system used to analyze and manipulate binary variables (0 and 1) using logical operations (AND, OR, NOT, XOR). Boolean algebra deals with logical operations that evaluate the truth values of statements, making it suitable for expressing and simplifying logical conditions.



BINARY

Binary is a base-2 numeral system that represents numbers using only two symbols, 0 and 1. Binary arithmetic involves mathematical operations (addition, subtraction, multiplication, division) performed with binary numbers.

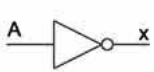
Introduction to Logic Gates

Logic gates are fundamental electronic components that process binary information (0s and 1s) based on Boolean algebra.

Types of Logic Gates:

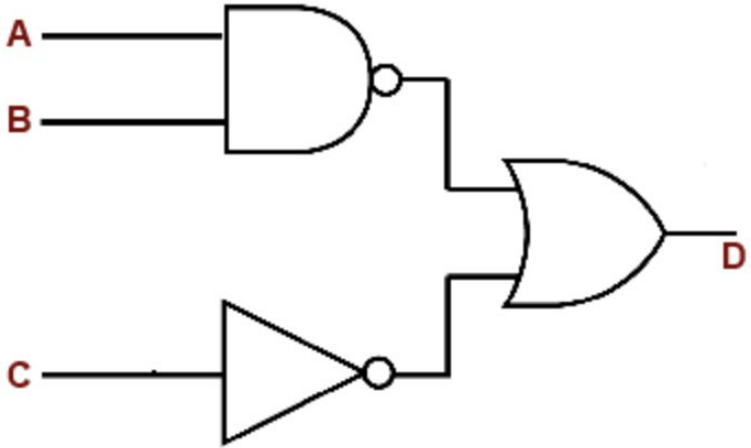
- **AND Gate**: Outputs 1 if both inputs are 1.
- **OR Gate**: Outputs 1 if at least one input is 1.
- **NOT Gate**: Inverts the input (1 becomes 0, and vice versa).
- **XOR Gate**: Outputs 1 if inputs are different.

Logic Gates

Name	NOT	AND	NAND	OR	NOR	XOR	XNOR
Alg. Expr.	$\overline{A}$	AB	$\overline{AB}$	$A+B$	$\overline{A+B}$	$A \oplus B$	$\overline{A \oplus B}$
Symbol							
Truth Table	A X 0 1 1 0	B A X 0 0 0 0 1 0 1 0 0 1 1 1	B A X 0 0 1 0 1 1 1 0 1 1 1 0	B A X 0 0 0 0 1 1 1 0 1 1 1 1	B A X 0 0 1 0 1 0 1 0 0 1 1 0	B A X 0 0 0 0 1 1 1 0 1 1 1 0	B A X 0 0 1 0 1 0 1 0 0 1 1 1

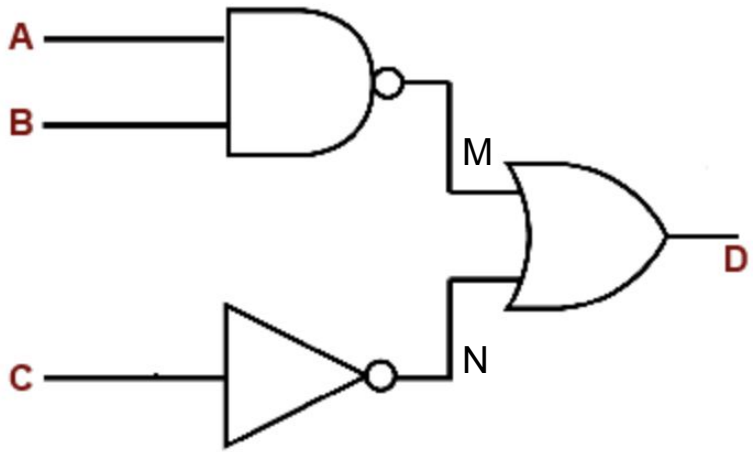
Tutorial Exercise on Logic Gates

Write down the logic statement and complete the truth table for the given logic circuit



Tutorial Exercise on Logic Gates

Write down the logic statement and complete the truth table for the given logic circuit



D=M+N

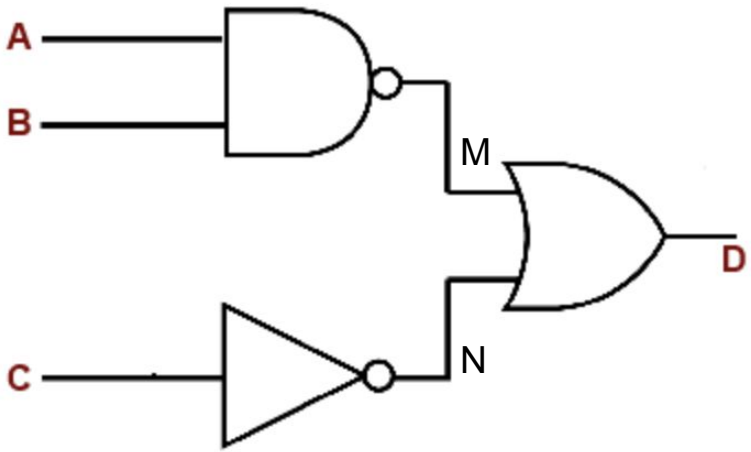
M=$\overline{AB}$

N=$\overline{C}$

D=($\overline{AB}$)+$\overline{C}$

Tutorial Exercise on Logic Gates

Write down the logic statement and complete the truth table for the given logic circuit



D=M+N

M=$\overline{AB}$

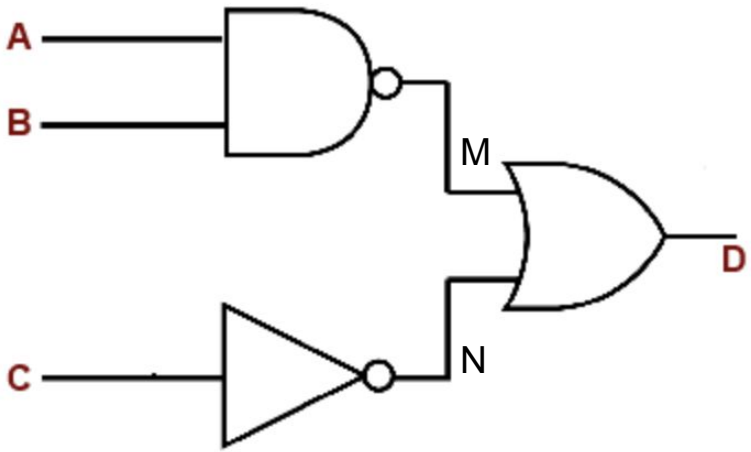
N=$\overline{C}$

D=$(\overline{AB})+\overline{C}$

A	B	C	D
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

Tutorial Exercise on Logic Gates

Write down the logic statement and complete the truth table for the given logic circuit



D=M+N

M=$\overline{AB}$

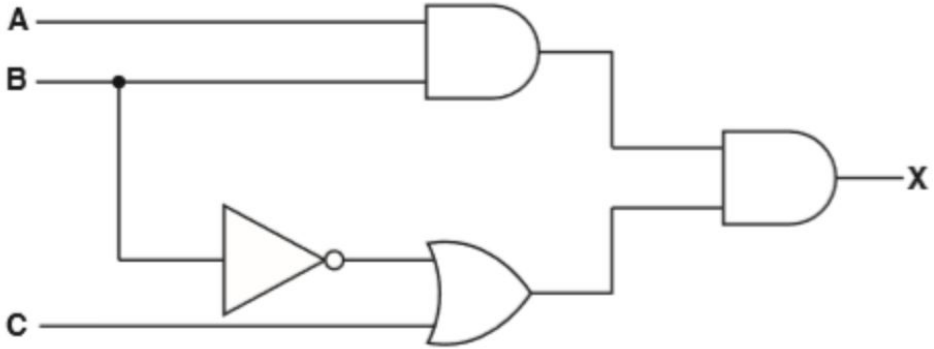
N=$\overline{C}$

D=($\overline{AB}$)+$\overline{C}$

A	B	C	D
0	0	0	**1**
0	0	1	**1**
0	1	0	**1**
0	1	1	**1**
1	0	0	**1**
1	0	1	**0**
1	1	0	**1**
1	1	1	**0**

Exercise 1 on Logic Gates

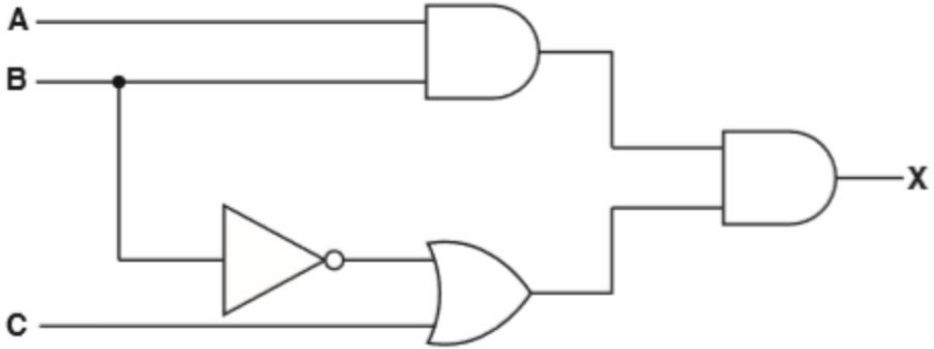
Write a logic statement that corresponds with the following logic circuit.



X=

Exercise 1 on Logic Gates

Write a logic statement that corresponds with the following logic circuit.



$X=(AB)(\bar{B}+C)$

Exercise 2 on Logic Gates

Given the previous logic circuit and its statement, write down the corresponding truth table

$$X=(AB)(\bar{B}+C)$$

A	B	C	X
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

Exercise 2 on Logic Gates

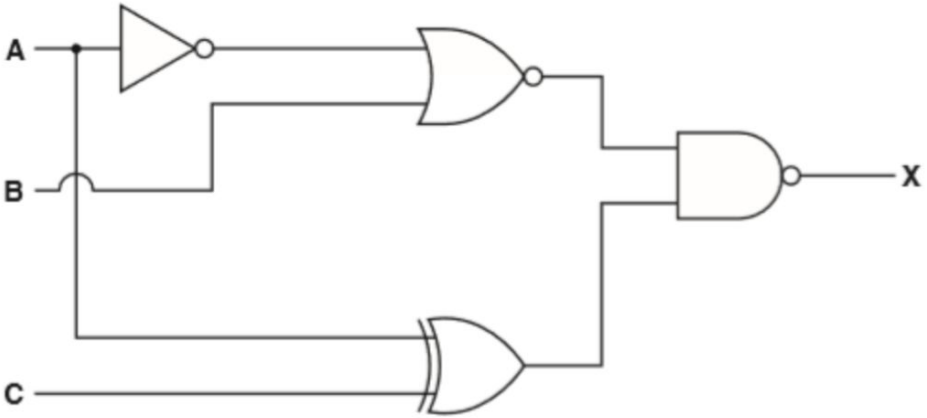
Given the previous logic circuit and its statement, write down the corresponding truth table

$$X=(AB)(\bar{B}+C)$$

A	B	C	X
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

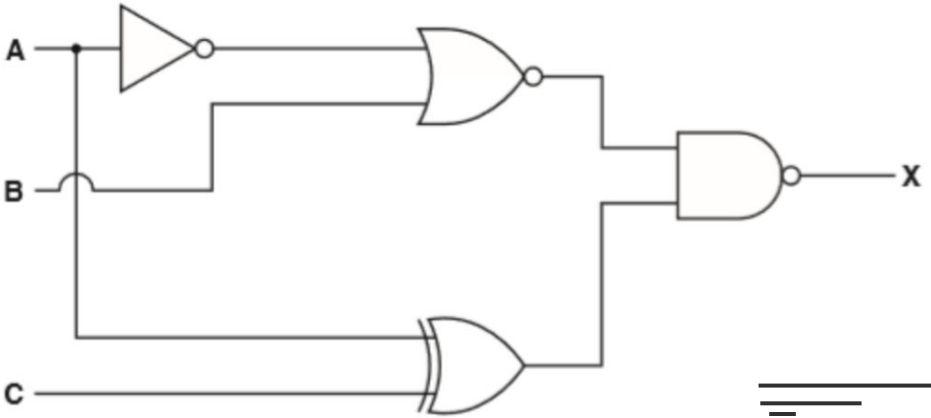
Exercise 3 on Logic Gates

Write down the logic statement for the given logic circuit



Exercise 3 on Logic Gates

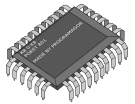
Write down the logic statement for the given logic circuit



$$X=\overline{(\overline{A}+B)}(A + C)$$

Programming Languages

Three main programming languages:

- Python 
- C and C++ 
- VHDL and Verilog (for FPGAs only) 

Python

Python is a versatile and readable programming language, well-suited for tasks like data analysis, automation, and scripting.

Pros:

- Versatile and readable.
- Rapid development and prototyping.
- Extensive libraries and frameworks.

Cons:

- Slower execution compared to low-level languages.
- May require more memory.

Hardware compatibility:

- Python is used in a variety of development boards and microcontrollers, including Raspberry Pi, Arduino, and more.

C and C++

C and C++ are programming languages known for their exceptional speed and efficiency, making them suitable for low-level operations, real-time control, memory management, and balancing performance and code organization.

Pros:

- Exceptional speed and efficiency.
- Direct hardware interaction.
- Real-time control capabilities.
- Extensive community support.

Cons:

- Steeper learning curve.
- More prone to errors.

Hardware compatibility:

- Wide range of microcontrollers and development boards, such as Arduino, STM32, and Raspberry Pi.

VHDL and Verilog

VHDL and Verilog are specialized hardware description languages exclusively used for configuring FPGAs. These languages are not intended for general-purpose programming but rather for precise hardware control.

Pros:

- Specialized for hardware description.
- Allows precise control of FPGA logic.
- Ideal for digital design and FPGA configuration.

Cons:

- Not suitable for general-purpose programming.
- Requires specific knowledge of FPGA hardware.

Hardware compatibility:

- VHDL and Verilog are exclusively used for configuring Field-Programmable Gate Arrays (FPGAs).

Introduction to Coding

Definition: Coding is the process of instructing a computer to perform a specific task by providing it with a set of step-by-step instructions.

Basic Syntax

A program consists of a series of instructions that the computer executes sequentially.
Essential components of a program:

- **Header Files**: Explain that header files like #include <stdio.h> are used to include libraries that provide essential functions and features.
- **main() Function**: Mention that every C program starts with the main() function, which serves as the entry point for the program.
- **Statements and Semicolons**: Emphasize that statements (lines of code) are terminated with semicolons (;) to indicate the end of an instruction.

Basic Syntax

```c
#include <stdio.h>

int main() {
    // This is a comment
    printf("Hello, World!"); // Print a message
    return 0; // Return 0 to indicate successful execution
}
```

Variables and Data Types

Variables are “containers” for storing data in a program.

Data types:

- **int**: For integer values (e.g., 42)
- **float**: For floating-point numbers (e.g., 3.14)
- **char**: For single characters (e.g., 'A') Explain the importance of choosing the right data type for efficiency and correctness. Mention naming conventions for variables, such as starting with a letter and using underscores (e.g., my_variable).

Variables and Data Types

Example:

```
int age = 25;
float temperature = 98.6;
char grade = 'A';
```

Control Structures

Control structures allow you to make decisions and perform repetitive tasks in a program:

- if-else statements: Describe conditional statements and how they allow you to execute code based on conditions.

```
if (age >= 18) {
    printf("You are an adult.");
} else {
    printf("You are a minor.");
}
```

- Loops (While and For).

Control Structures

- While Loop: Repeats a block of code as long as a specified condition is true.

```
while (condition) {
    // Code to repeat
}
```

- For Loop: Used when you know the number of iterations in advance.

```
for (initialization; condition; increment/decrement) {
    // Code to repeat
}
```

Functions

A function is a reusable block of code that performs a specific task.

Function components:

- Declaration
- Definition
- Parameters and Return Values

```
// Function declaration
int add(int a, int b);

// Function definition
int add(int a, int b) {
    return a + b;
}
```

Functions

A function is a reusable block of code that performs a specific task.

Function components:

- Declaration
- Definition
- Parameters and Return Values

```
// Function declaration
int add(int a, int b);

// Function definition
int add(int a, int b) {
    return a + b;
}
```

Functions

Declaration: tells the compiler about the function's name, return type, and the types of its parameters (but not their names). A function declaration is necessary when you want to use a function before its actual definition in your code.

```
int add(int a, int b);   // Function declaration
```

Definition: it includes the function's name, its return type, the parameter list (if any), and the code block that specifies what the function does. In a function definition, you write the actual code that gets executed when the function is called.

```
int add(int a, int b) {   // Function definition
    return a + b;
}
```

Functions

Parameters: Parameters allow you to pass data into a function so that it can perform its task with specific values. In the function definition, you provide names for the parameters, and these names are used within the function's code block to work with the passed-in values.

```
int add(int a, int b) {   // 'a' and 'b' are parameters
    return a + b;
}
```

Return Values: values that a function sends back to the code that called it. When a function is called, it may perform some computations and return a result using the return statement. The return value can be of any data type, depending on the function's declaration.

```
int add(int a, int b) {   // 'int' indicates the return type
    return a + b;         // The result of 'a + b' is returned
}
```

Let’s Code!

Copy the following link and play!

https://codecombat.com

How to choose an OBC

CPU Clock Speed:

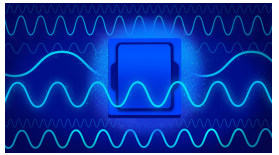
Suppose your CubeSat mission requires a specific data processing speed, such as processing sensor data at a rate of 500,000 data points per second. Let's assume each data point requires 50 clock cycles for processing. Here's how you can calculate the required CPU clock speed:

- Data Points Processed Per Second = 500,000
- Clock Cycles per Data Point = 50

The required CPU clock speed (in Hz) can be calculated as:

- Required CPU Clock Speed = (Data Points Processed Per Second) × (Clock Cycles per Data Point)
- Required CPU Clock Speed = 500,000 data points/second × 50 clock cycles/data point = 25,000,000 Hz = 25 MHz

So, in this example, you would need a CPU with a clock speed of at least 25 MHz to meet the data processing requirements of your mission.



How to choose an OBC

RAM:

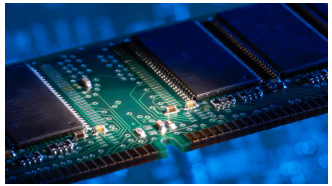
Let's assume your CubeSat mission needs to store a set of sensor measurements in RAM for real-time analysis. Each measurement set occupies 256 KB of RAM, and you want to store 20 sets simultaneously. Here's how you can calculate the required RAM size:

- Size of Each Measurement Set = 64 KB
- Number of Sets in RAM = 2

The required RAM size (in KB) can be calculated as:

- Required RAM Size = (Size of Each Measurement Set) × (Number of Sets in RAM)
- Required RAM Size = 64 KB/set × 2 sets = 5,120 KB = 128 KB

So, in this example, you would need at least 128 KB of RAM to store the required sensor measurements for real-time analysis.



How to choose an OBC

SSD Memory Size:

Suppose your CubeSat mission has the following parameters:

- Data Storage Rate: 16 MB/hour (data generated by environmental sensors and cameras).
- Mission Duration: 365 days (approximately 8,760 hours).
- Data Retention Factor: 1.5 (to account for redundancy and margin).

Required SSD Size (GB) = (Data Storage Rate * Mission Duration) / (1024 MB/GB) * Data Retention Factor

Required SSD Memory Size (GB) = (16 MB/hour * 8,760 hours) / (1024 MB/GB) * 1.5 = 192.83 GB

How to choose an OBC

Power Consumption:
In CubeSat missions, On-Board Computers (OBCs) serve as vital components, but their power-intensive nature necessitates careful consideration.
Their power consumption significantly impacts the mission's overall power budget, making efficient power management critical.
Striking the right balance between functionality and power constraints demands thoughtful OBC selection and configuration.
When choosing an OBC, a low power consumption is considered a great advantage, sometimes even better than memory and CPU specifications.

How to choose an OBC

Interface Requirements:

UART (Universal Asynchronous Receiver-Transmitter):

- You plan to communicate with various onboard sensors and instruments using UART for data acquisition and control.
- Specify the baud rates, data formats, and pin connections for UART communication with specific sensors.

I2C (Inter-Integrated Circuit):

- You intend to connect I2C devices, such as environmental sensors, memory modules, or peripheral devices, for data exchange and control.
- Provide details on I2C addresses, bus speeds, and the purpose of each connected device.

SPI (Serial Peripheral Interface):

- SPI is used for high-speed data transfer with certain devices like memory chips, attitude control systems, or other peripherals.
- Specify the SPI modes (e.g., clock polarity and phase), slave select lines, and data rates for connected devices.

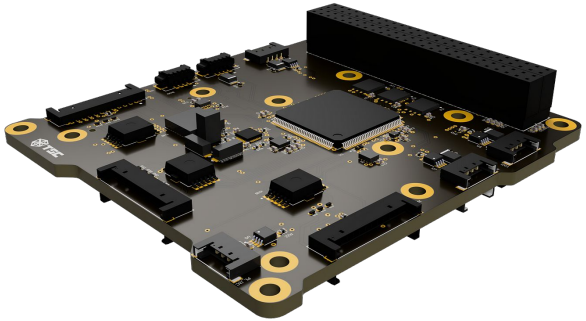
GPIO (General-Purpose Input/Output):

- GPIO pins can be configured for various purposes, including controlling external components or receiving digital signals.
- Detail the specific GPIO pins used, their functions, and voltage levels.

How to choose an OBC

Example configuration:

- Processor: e.g. ARM Cortex M4
- CPU Clock Speed: e.g. 100 MHz
- RAM Memory: e.g. 128 KB
- SSD Memory: 256GB
- Flash Memory: e.g. 256KB
- Power Consumption: e.g. 1W
- Interfaces: e.g. UART, I2C, SPI, GPIO



Limitations of OBDHs & OBCs

Common challenges that engineers face when designing OBDHs and OBCs:

HARDWARE:

- Limited power and processing speed
- Radiation vulnerability
- Sensors reliability

SOFTWARE:

- Data volume management
- Latency challenges
- Complexity in autonomous operations
- Compatibility issues
- Software maintenance.

Limitations of OBDHs & OBCs

Common challenges that engineers face when designing OBDHs and OBCs:

HARDWARE:

- Limited power and processing speed
- **Radiation vulnerability**
- Sensors reliability

SOFTWARE:

- Data volume management
- Latency challenges
- Complexity in autonomous operations
- Compatibility issues
- Software maintenance.

Radiation Vulnerability

Radiation in space can affect a Cubesat On-Board Computer and On-Board Data Handling system in several ways:

- Data Corruption: High-energy particles in space can strike OBC and OBDH components, causing transient errors or "bit flips" in memory and data registers. This radiation-induced interference can corrupt data, potentially leading to incorrect calculations or system malfunctions.
- Single-Event Effects (SEEs): Radiation can trigger SEEs, which are sudden, one-time disruptions in electronic circuits. This may result in temporary glitches or even permanent damage to components within the systems.
- Degrading Components: Over time, exposure to radiation can degrade the performance and reliability of OBC and OBDH components. This degradation may reduce the lifespan and effectiveness of the systems.

Radiation Vulnerability: Workarounds

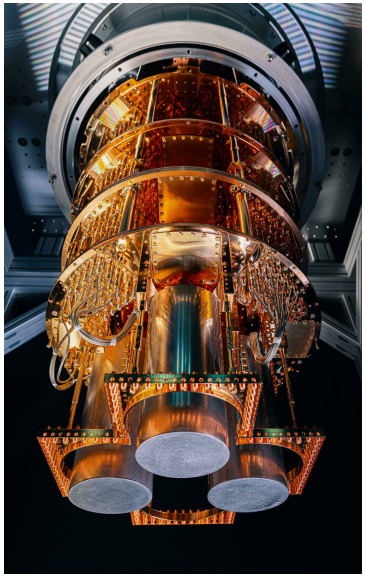
Through various strategies and technologies, engineers can ensure that Cubesats are capable of enduring and resisting radiation during space missions:

- Radiation-Hardened Components: Specially designed parts that resist radiation damage.
- Error-Detection Software: Intelligent software that spots and fixes radiation-induced errors.
- Redundancy and Backups: Spare systems that take over if radiation affects primary ones.
- Shielding and Armor: Protective layers that block harmful radiation.
- Rigorous Testing: Extensive simulations to prepare for radiation challenges.

Innovative Technologies in OBDHs

- Machine Learning
- AI-Based Decision Making
- Advanced Data Compression
- Quantum Computing

Innovative Technologies in OBCs



- Multi-Core Processors
- Neuromorphic Computing
- Quantum Computing
- Swarm Intelligence

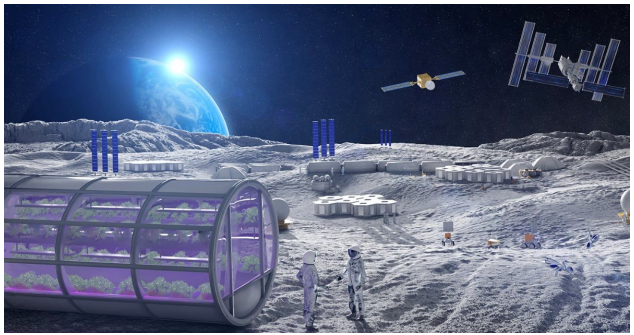
Conclusion

In summary, this presentation showcased the vital roles of OBCs and OBDHs in CubeSat missions. These systems are the digital brains behind compact satellites, driving mission success in space.

We explored their fundamentals, dimensions, and innovative technologies. Real-world examples illustrated their significance in CubeSat missions.

While CubeSats face challenges, ongoing advancements promise solutions.

In essence, OBC and OBDH are essential for CubeSat missions, propelling space missions in smaller packages. The future holds exciting possibilities.



Q&A